

Programmation scientifique : Déplacement tectonique de stations GNSS

Matteo CAPOZZA, Paloma PIRES–GODARD

Janvier 2026

Table des matières

1	Partie 1 : Lecture et formatage des données (partie_1.py)	3
1.1	Fonction <code>read_gnss()</code>	3
1.2	Fonctions <code>create_gem_file()</code> et <code>read_gem()</code>	3
1.3	Fonction <code>read_plates()</code>	4
1.4	Sortie graphique	4
2	Partie 2 : Appartenance à une plaque tectonique (partie_2.py)	5
2.1	Fonction <code>ray_casting()</code>	5
2.2	Fonction <code>find_plate_for_row()</code>	5
2.3	Fonction <code>which_plate()</code>	5
2.4	Sortie graphique	5
3	Partie 3 : Proximité à une zone de déformation (partie_3.py)	7
3.1	Fonction <code>haversine()</code>	7
3.2	Fonctions <code>check_station()</code> et <code>calculate_in_deformation()</code>	7
3.3	Sortie graphique	7
4	Partie 4 : Détermination des déplacements de stations (partie_4.py)	8
4.1	Fonction <code>cartesian_to_angular_velocities()</code> :	8
4.2	Fonction <code>v_predite()</code>	8
4.3	Fonction <code>calculate_v_pred()</code>	8
4.4	Analyse des vitesses prédites par le modèle de plaques ITRF2020-PMM	8
4.5	Sorties graphiques	9
5	Partie 5 : Représentations cartographiques (partie_5.py)	10

Le sujet : Le déplacement tectonique de stations GNSS

Notre position exacte sur la Terre peut être calculée à chaque instant grâce aux stations GNSS, dont les coordonnées sont connues et situées dans un repère géodésique de référence commun : l'International Terrestrial Reference Frame (ITRF). Cependant, la surface de la Terre n'est pas figée : les plaques lithosphériques se déplacent lentement et modifient la surface de la Terre continuellement. Ces déplacements provoquent également le déplacement des stations GNSS, il faut donc recalculer régulièrement l'ITRF pour prendre en compte ces déplacements.

Le sujet invite à étudier le déplacement des stations GNSS en prenant en compte leurs vitesses de déplacement sur les plaques (issu du comportement rigide des plaques) ainsi que leurs positions par rapport aux zones de déformation (déformations issues du comportement local des plaques). En effet, si les vitesses de déplacement sont mesurées par les stations GNSS elles mêmes, on peut également prédire la vitesse de déplacement des stations grâce au modèle ITRF2020 Plate Motion Model (PMM). Grâce au vecteur de rotation, on peut prédire la vitesse théorique d'un point sur la plaque et on pourra donc comparer ces vitesses prédites et celles observées par les stations GNSS.

On peut cependant noter que les stations GNSS ne sont pas présentes de manière équivalente sur tous les continents : certaines zones du globe, comme l'Europe, sont plus dotées en stations que d'autres, ce qui peut rendre l'analyse incomplète. De plus, le modèle ITRF2020 Plate Motion Model (PMM), qui fournit les pôles de rotation et vitesses angulaires des 13 principales plaques tectoniques omet les plaques plus petites, ce qui empêche également une analyse complète des déplacements des plaques terrestres.

Pour ce faire, on procède en 4 étapes : après avoir lu et ajusté (formatage et conversion de certaines valeurs) les différents jeux de données, on associe chaque station GNSS à la plaque sur laquelle elle est située, car le modèle ITRF2020 Plate Motion Model (PMM) ne donne pas d'informations sur certaines plaques, et donc sur les vitesses des stations GNSS situées sur ces plaques. De plus, ce modèle n'est pas valable pour les stations situées en zone de déformation car leurs vitesses sont bien différentes de celles des stations situées sur le reste des plaques, il faut donc filtrer ces stations. On calcule enfin les déplacements des stations se trouvant hors zone de déformation, et on compare ces vitesses aux vitesses observées.

Pour rédiger ce rapport, nous avons choisi d'utiliser le site Overleaf, sur lequel il est possible de programmer en $\text{\LaTeX}$ tout en partageant le document, ce qui nous a permis d'écrire ce rapport en commun.

1 Partie 1 : Lecture et formatage des données (partie_1.py)

L'objectif de la partie 1 est de lire les trois fichiers :

- `ITRF2020_GNSS.SSC.txt` : les coordonnées des stations GNSS ITRF2020
- `GSRM_strain.txt` : le modèle de déformation (GEM Strain Rate Model)
- `Tectonic_Plates.geojson` : les limites de plaques tectoniques

Notre code est organisé en deux parties : les fonctions qui contiennent les formules de transformation des coordonnées cartésiennes géocentriques en coordonnées géographiques et de calcul de la magnitude de déformation ; les cinq fonctions qui permettent de lire et de créer les DataFrames adaptés pour les quatre fichiers.

1.1 Fonction `read_gnss()`

Objectif : Lecture, filtrage et calculs pour le fichier `ITRF2020_GNSS.SSC.txt`.

	DOMES_NB	SITE_NAME	TECH	ID	X_Vx	...	DATA_END
0	10001S006	Paris	GNSS	OPMT	4202777.2446	...	
1	10001S006				-.01278	...	
2	10001S007	Paris	GNSS	OP71	4202779.7959	...	
3	10001S007				0.01047	...	
4	10002M006	Grasse (OCA)	GNSS	GRAS	4581690.7636	...	96 :277 :00000
5	10002M006				-.01364	...	
:	:	:	:	:	:	...	:

TABLE 1 – Aperçu du fichier `ITRF2020_GNSS.SSC.txt` avant filtrage et séparation des DFs

Nous avons utilisé la fonction `read_fwf` de la librairie **Pandas** pour créer un Data Frame à partir du fichier texte car les colonnes de ce fichier sont irrégulières, et on les délimite manuellement. On crée ensuite deux nouveaux Data Frame à partir du premier, l'un contenant les coordonnées des stations et l'autre les vitesses de chaque station en séparant les colonnes `X/Vx`, `Y/Vy` et `Z/Vz` en `X`, `Y` et `Z` pour le Data Frame `df_gnss_c` contenant les coordonnées et `Vx`, `Vy` et `Vz` pour le Data Frame `df_gnss_s` contenant les vitesses. Pour cela, on utilise les méthodes `isna` et `notna`. On réutilise la méthode `isna` pour filtrer le Data Frame et supprimer les lignes contenant des informations sur une station qui n'est plus active. Lors de la création de nos deux nouveaux DataFrames, on a supprimé les colonnes contenant les indices, ce qui nous permet maintenant d'associer les nouveaux indices du DataFrame `df_gnss_c` à ceux du DataFrame `df_gnss_s` et on peut grâce à cela supprimer facilement dans le Data Frame `df_gnss_s` les lignes à propos de stations qui ne sont plus actives.

On ajoute enfin quatre colonnes dans le DataFrame contenant les coordonnées : `lamb_rad`, `phi_rad`, `lamb_deg` et `phi_deg` grâce à la fonction de conversion `xyz_to_llh()`, et en ayant au préalable converti les informations contenues dans les colonnes `X`, `Y` et `Z` en flottants.

1.2 Fonctions `create_gem_file()` et `read_gem()`

Objectif : Lecture, filtrage et calculs pour le fichier `GSRM_strain.txt`.

On utilise la même méthode que dans la fonction `read_gnss()` pour lire le fichier et créer le DataFrame. Grâce à la fonction `strain_magnitude` créée plus haut dans le code, on crée une nouvelle colonne dans le DataFrame contenant la magnitude de déformation et on ne conserve que les points ayant une déformation significative, c'est-à-dire les points avec une magnitude de déformation supérieure à 50 nanostrain/an. On trie ensuite cette colonne `strain_mag` par ordre décroissant et on supprime les doublons en ne conservant que la magnitude de déformation la plus haute en utilisant les méthodes `sort_values` et `drop_duplicates`.

On choisi ensuite de convertir le Data Frame créé en un fichier CSV `GEM_filtered.csv`, car le DataFrame original contient plus de 3 millions de lignes. La fonction `read_gem()` permet de lire ce fichier CSV et de créer un nouveau DataFrame en utilisant la fonction `read_csv` de la librairie `Pandas`. Dans le reste du projet, on lira directement ce fichier filtré, ce qui réduit largement le temps d'exécution du code (de 50s à 2s).

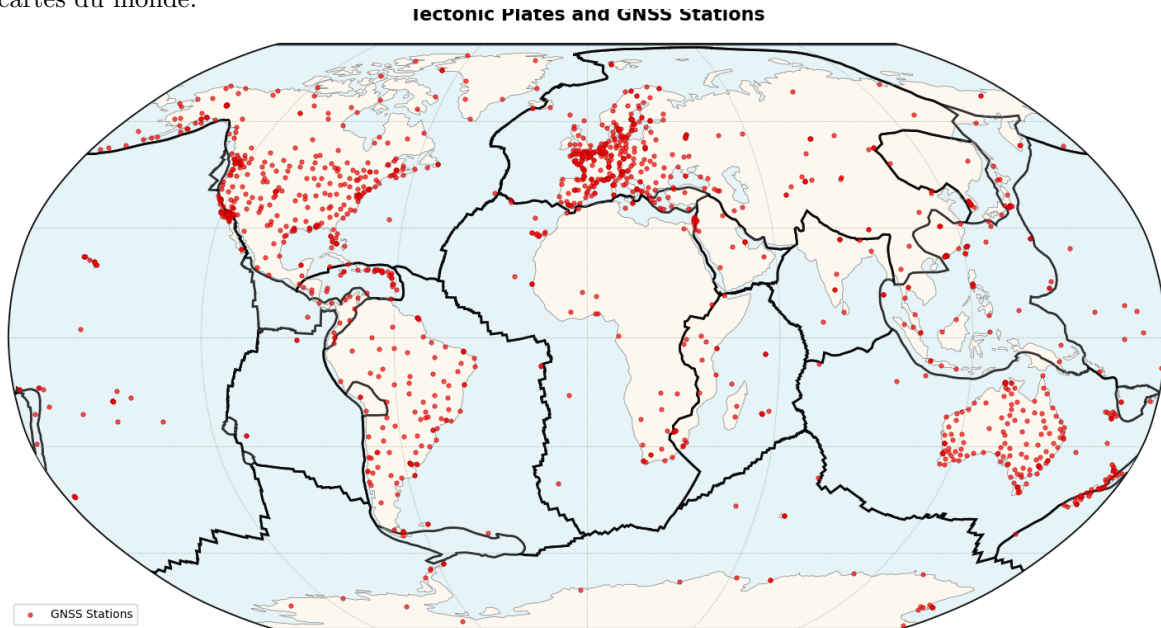
1.3 Fonction `read_plates()`

Objectif : Lecture, filtrage et calculs pour le fichier `Tectonic_Plates.geojson`.

Pour lire ce fichier GeoJSON, on utilise la fonction `load` de la librairie `json`. On créé le dictionnaire `dict_plates` qui va rassembler les 13 plaques dans le fichier `pmm.itrf.txt` (que nous avons chargé au préalable dans un Data Frame en faisant attention à la différence d'écriture de "North America" et "South America" entre ce fichier et le fichier GeoJSON). Pour chaque plaque dans le dictionnaire, on y associe une liste de Data Frame appelée `plate_polygons` qui contient les coordonnées qui permettent de tracer les polygones formant les plaques. Après avoir extrait les noms des plaques, leurs types de géométrie et les coordonnées du polygone de chaque plaque grâce à l'attribut `features` des fichiers GeoJSON. Nous avons délibérément fait le choix d'une liste de DataFrame pour gérer les multipolygones plutôt qu'un seul DataFrame délimitant les différentes géométries par des Nan car nous avons jugés cette méthode plus "propre".

1.4 Sortie graphique

La lecture et le formatage des données ne permettent pas à elles seules d'obtenir des sorties graphiques nombreuses, mais on peut cependant d'hors et déjà représenter sur une carte les différentes plaques tectoniques ainsi que la position des stations GNSS dans le monde. Cette carte permet de visualiser facilement l'inégale répartition des stations GNSS dans le monde et permet de voir quelles stations seront situées sur des zones de déformations, qui sont souvent des limites de plaques. On comprend également pourquoi certaines plaques, comme la plaque australienne, sont des multi polygones : elles sont divisées dans l'océan Pacifique par la projection choisie ici et traditionnellement pour les cartes du monde.



2 Partie 2 : Appartenance à une plaque tectonique (partie_2.py)

Dans cette partie, on cherche à associer chaque station GNSS à la plaque tectonique à laquelle elle appartient. On utilise pour cela la méthode de Ray Casting qui permet de déterminer si un point appartient à un polygone ou non. Ici, nos points sont nos stations GNSS dont on a les coordonnées et nos polygones nos limites de plaques dont on a également les coordonnées. Notre code se divise en trois parties : la fonction `ray_casting()` qui renvoie si oui ou non un point appartient à un polygone ; la fonction `find_plate_for_row()` qui retourne le nom de la plaque d'appartenance d'un point (row donné) ; et la fonction `which_plate()` qui associe à chaque point du DataFrame la plaque sur laquelle il se situe.

2.1 Fonction `ray_casting()`

Objectif : Déterminer si un point appartient ou non à un polygone.

On crée d'abord une boîte englobante pour ne pas avoir à calculer l'appartenance pour tous les points à chaque fois, ce qui permet de réduire le temps d'exécution de notre algorithme : on trace un carré avec des coordonnées `xmin`, `xmax`, `ymin`, `ymax` qui délimitent la zone où le point pourrait être : en dehors de ce carré, le point est forcément hors du polygone. Dans le cas où le point est dans la boîte, il faut vérifier si ce point est bien dans le polygone ou non, pour cela, on crée un compteur d'intersections, si le nombre d'intersections est impair : le point est à l'intérieur du polygone. On ne trace pas réellement de ligne depuis le point à tester mais on calcule la coordonnée en x du point d'intersection entre cette demi-droite imaginée et les limites du polygone. Enfin, si la coordonnée en x du point considéré est inférieure à la coordonnée en x du point d'intersection, on considère qu'une intersection a eu lieu car on ne compte que les intersections avec les plaques à l'est du point. On finit par déterminer si le point appartient ou non au polygone en calculant si le nombre total d'intersections comptabilisées est pair ou non : s'il est pair, le point n'appartient pas au polygone (False) et s'il est impair, le point appartient au polygone (True).

2.2 Fonction `find_plate_for_row()`

Objectif : Trouver la plaque d'appartenance d'un point.

Cette fonction fait appel à la fonction `ray_casting()` et retourne pour un row de DataFrame donné la plaque d'appartenance d'un point.

2.3 Fonction `which_plate()`

Objectif : Associer à chaque station GNSS du DataFrame la plaque à laquelle elle appartient

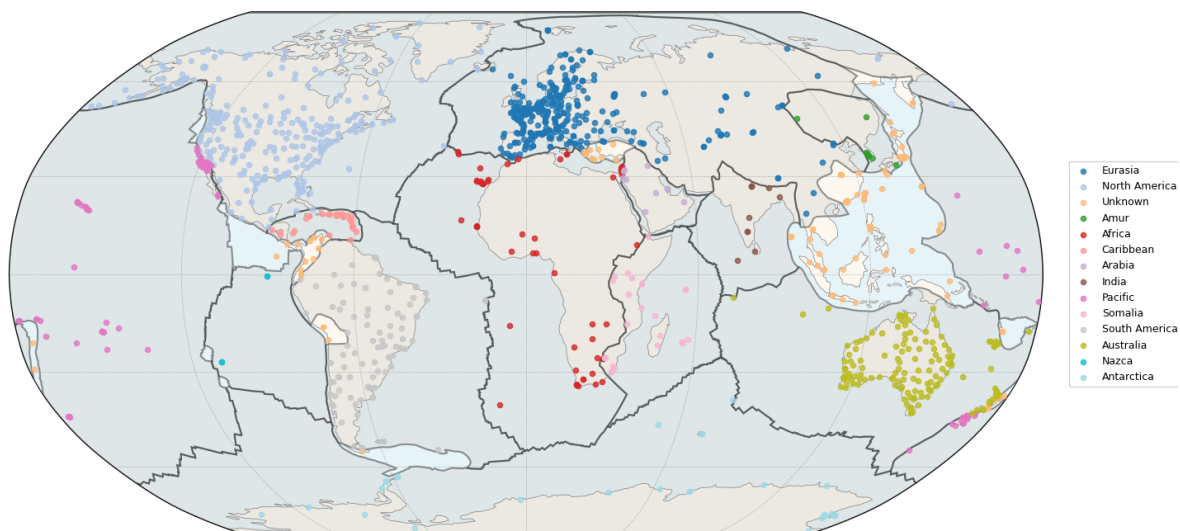
Dans cette fonction, on réutilise les deux fonctions précédentes afin d'aboutir au résultat final souhaité. Afin d'optimiser le temps d'exécution on ne calcule la boîte englobante d'une plaque qu'une seule fois plutôt que pour chaque appel de la fonction de Ray Casting (autant que le nombre de points). On ajoute ainsi une nouvelle colonne au DataFrame appelée 'plate' qui comprend le nom de la plaque auquel appartient chaque station.

2.4 Sortie graphique

Nous pouvons désormais représenter les stations GNSS en fonction des plaques sur lesquelles elles sont situées. La représentation graphique de cette partie nous permet de vérifier très facilement que nous n'avons pas fait d'erreurs lors de l'implémentation de nos fonctions. Ici, toutes les stations GNSS sont bien aux couleurs des plaques sur lesquelles elles sont situées. On peut également noter que les

13 plaques qui font partie du modèle ITRF2020-PPM sont plus opaques que les plaques qui n'en font pas partie pour marquer les données qui ne nous servons pas dans la suite du projet.

GNSS Stations Colored by Tectonic Plate



3 Partie 3 : Proximité à une zone de déformation (partie_3.py)

3.1 Fonction haversine()

Objectifs : Calculer la distance haversine entre 2 points.

La formule de Haversine permet de calculer la distance la plus courte entre deux points à la surface d'une sphère, en tenant compte de la courbure de la Terre. La formule est la suivante :

$$d = 2R_T \arcsin\left(\sqrt{\sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right)$$

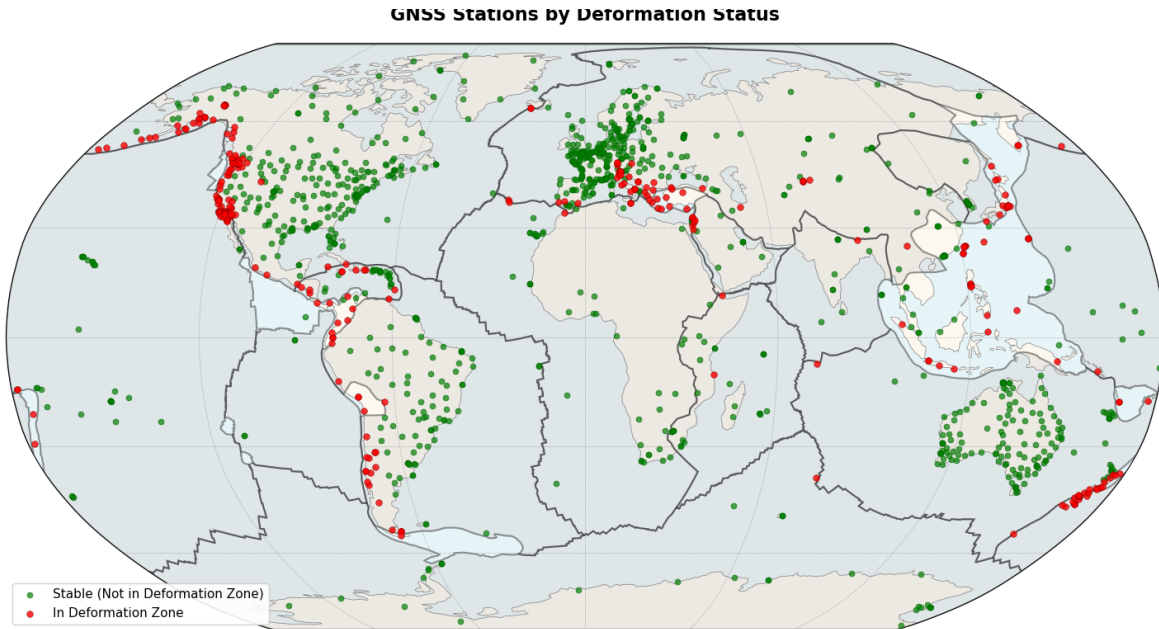
3.2 Fonctions check_station() et calculate_in_deformation()

Objectifs : Ajouter au `df_gnss_c` une colonne avec pour chaque station True si elle est dans une zone de déformation, False sinon.

La fonction `check_station()` a pour but de déterminer si une station GNSS donnée se situe dans une zone de déformation (à moins de 50 km d'un point GEM). Pour ce faire elle utilise la fonction `haversine()` et l'applique aux coordonnées de la station pour l'ensemble des points GEM, la vectorisation de cette opération est bien plus rapide qu'un appel ligne par ligne. La fonction `calculate_in_deformation()` parcourt l'entièreté du `df_gnss_c` et applique la fonction `check_station()` à chaque ligne du Data Frame.

3.3 Sortie graphique

La représentation graphique permet encore une fois de vérifier de manière visuelle et rapide la justesse de nos fonctions. On remarque que la grande majorité des stations situées dans des zones de déformation sont celles situées à des frontières de plaques, ce qui cohérent car les zones où les plaques s'entrechoquent et se croisent sont celles où la déformation est la plus forte.



4 Partie 4 : Détermination des déplacements de stations (partie_4.py)

Dans cette partie, on cherche à calculer les vitesses de déplacement de toutes les stations GNSS en fonction du déplacement des plaques tectoniques sur lesquelles elles sont situées. Pour cela, on utilise le modèle cinématique des plaques rigides ITRF2020-PMM qui comprend les 13 plaques étudiées jusqu'alors. Cette partie comprend 3 fonctions :

- `cartesian_to_angular_velocities()`
- `v_predite()`
- `calculate_v_pred()`

4.1 Fonction `cartesian_to_angular_velocities()` :

Objectif : Transformer des vitesses cartésiennes en vitesses angulaires

Pour pouvoir afficher sur une carte les vecteurs vitesses que nous allons calculer pour chaque station GNSS, nous devons au préalable transformer les vitesses cartésiennes V_x , V_y et V_z en vitesses angulaires. On projette les vitesses cartésiennes dans le repère local : Est, Nord, Vertical (East, North, Up) à partir des angles géographiques λ (longitude) et Φ (latitude) puis on fait une approximation, suffisante pour la représentation graphique des vecteurs vitesse, afin obtenir des vitesses angulaires qui peuvent être projetées sur une carte. On veille à convertir en degrés les angles obtenus en radians avant d'ajouter ces valeurs à notre Data Frame.

4.2 Fonction `v_predite()`

Objectif : Calculer la vitesse prédite de chaque station

Cette fonction permet de calculer de manière vectorisée la vitesse prédite d'après la formule suivante :

$$\vec{v}_{pred} = \vec{\omega} \times \vec{r} + \vec{T}$$

4.3 Fonction `calculate_v_pred()`

Pour diminuer le temps d'exécution de notre code, on fait attention à ne faire le calcul de la vitesse des stations que pour les stations hors des zones de déformation et les stations associées à une des 13 plaques du modèle. Pour effectuer le calcul vectoriel de la vitesse des stations, on fait attention de convertir les vitesses angulaires, qui sont fournies en milliarcsecondes/an en radians/an. Après avoir calculé la vitesse prédite grâce à la fonction `v_predite()`, on conserve les normes des vecteurs vitesse qui seront affichés sur la carte.

4.4 Analyse des vitesses prédites par le modèle de plaques ITRF2020-PMM

On observe que les 10 stations avec les vitesses les plus hautes sont toutes sur la plaque Pacifique. Ces 10 stations sont situées sur des îles qui ont été formées par la forte activité volcanique sur cette plaque, c'est donc sans doute encore aujourd'hui une plaque très active, et qui se déplace beaucoup. On observe également que sur les 10 stations avec les vitesses les plus faibles, 8 sont situées sur la plaque Antarctique. Cela fait sens dans un contexte géodésique : les plaques situés aux pôles se déplacent moins que les autres. De plus, le modèle ne fonctionne pas pour les stations situées sur des zones de déformation car dans ces cas là la vitesse de déplacement de ces stations ne suit pas la vitesse de déplacement des plaques, et doit être bien plus importante ou bien plus faible.

	DOMES_NB	SITE_NAME	TECH	ID	X	...	plate	in_deformation	v_pred	Vlamb_deg	Vphi_deg
1167	51301M001	Kiritimati	GNSS	KRTM	-5.886689e+06	...	Pacific	False	0.074704	-6.073477e-07	3.031708e-07
405	21789S001	Minamitorishima	GNSS	MCIL	-5.227188e+06	...	Pacific	False	0.074593	-7.045750e-07	2.194021e-07
1156	50506M001	KWAJALEIN ATOLL	GNSS	KWJ1	-6.160881e+06	...	Pacific	False	0.074252	-6.283469e-07	2.652976e-07
1170	51501M001	MAJURO	GNSS	MAJU	-6.257572e+06	...	Pacific	False	0.074243	-6.255281e-07	2.732782e-07
1315	92509M001	Makemo tide gau	GNSS	MKTG	-4.918470e+06	...	Pacific	False	0.073985	-6.325366e-07	3.009396e-07
1314	92501M002	Rangiroa	GNSS	RGTG	-5.210326e+06	...	Pacific	False	0.073935	-6.281499e-07	2.965243e-07
1312	92301M003	Rikitea (Ile Ma	GNSS	GAMB	-4.147127e+06	...	Pacific	False	0.073481	-6.612200e-07	2.859581e-07
1153	50305M001	BETIO ISLAND -	GNSS	KIRI	-6.327822e+06	...	Pacific	False	0.073299	-6.127797e-07	2.760763e-07
1308	92201M011	Papeete (Tahiti	GNSS	PAPE	-5.245799e+06	...	Pacific	False	0.073226	-6.246390e-07	3.069331e-07
1309	92201M012	Papeete (Tahiti	GNSS	FAA1	-5.247393e+06	...	Pacific	False	0.073216	-6.246524e-07	3.070596e-07

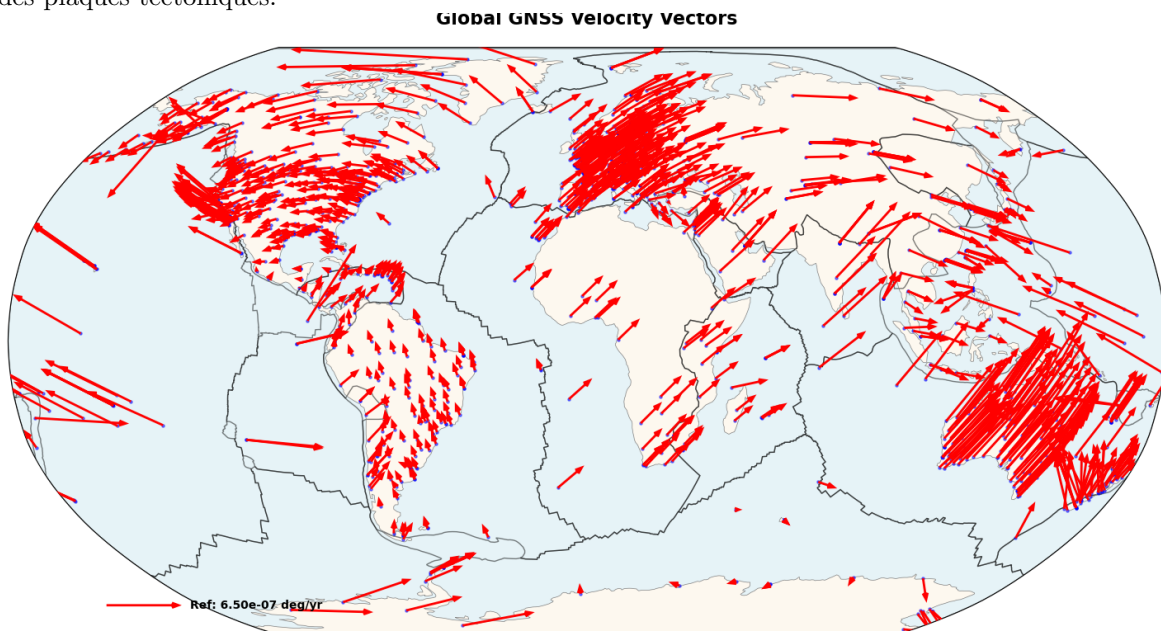
FIGURE 1 – Les 10 stations avec les vitesses prédites les plus hautes

	DOMES_NB	SITE_NAME	TECH	ID	X	...	plate	in_deformation	v_pred	Vlamb_deg	Vphi_deg
1254	66004M001	Mawson station	GNSS	MAW1	1.111287e+06	...	Antarctica	False	0.004524	-8.688291e-08	-2.029204e-08
1256	66006S002	Syowa	GNSS	SYOG	1.766208e+06	...	Antarctica	False	0.004662	-9.880535e-08	2.346554e-08
1303	91301M002	ILE DE LA POSSE	GNSS	CZTG	2.719995e+06	...	Antarctica	False	0.005332	6.230538e-08	3.367145e-10
1302	91201M009	Kerguelen	GNSS	KETG	1.408882e+06	...	Antarctica	False	0.006545	6.472614e-08	-3.417552e-08
1301	91201M007	Kerguelen	GNSS	KRGG	1.406336e+06	...	Antarctica	False	0.006552	6.475066e-08	-3.415454e-08
1300	91201M002	Kerguelen	GNSS	KERG	1.406337e+06	...	Antarctica	False	0.006552	6.475057e-08	-3.415454e-08
1262	66010M001	Davis	GNSS	DAVR	4.868546e+05	...	Antarctica	False	0.006668	-7.230083e-08	-4.846042e-08
1263	66010M001	Davis	GNSS	DAV1	4.868546e+05	...	Antarctica	False	0.006668	-7.230083e-08	-4.846042e-08
694	40902S001	Santa Elena	GNSS	ELEN	1.410371e+04	...	North America	False	0.008956	-6.695448e-08	7.957437e-09
806	42602M001	PedroCay_JAM201	GNSS	CN11	1.290838e+06	...	Caribbean	False	0.009105	8.430904e-08	6.835072e-08

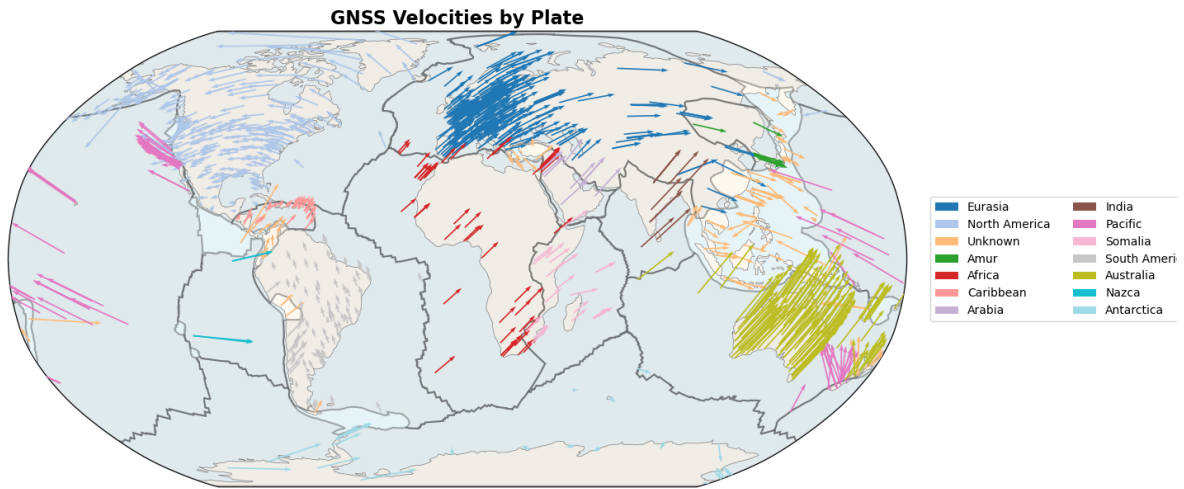
FIGURE 2 – Les 10 stations avec les vitesses prédites les plus faibles

4.5 Sorties graphiques

On représente les vecteurs de vitesses de déplacement des stations GNSS à cause du mouvement des plaques tectoniques.



Comme on connaît aussi l'appartenance de chaque station GNSS à une plaque, on choisit aussi de représenter les vecteurs vitesses de chaque station avec une couleur différente pour chaque plaque afin d'obtenir un rendu plus lisible.



5 Partie 5 : Représentations cartographiques (partie_5.py)

Pour créer nos cartes avec Python, on utilise les librairies `cartopy.crs` et `cartopy.feature`.